Apply the methods, approaches, and examples provided in the lectures and exercises to develop a technical **prototype** that features gesture classification in Python and enables slideshow control via gestures. **Comprehend** your prototype and be prepared to demonstrate your understanding by explaining the core aspects of its implementation.

The gestures to be recognized shall come from a live camera feed or video input. Mediapipe shall be used to extract the positions of body parts from video data. These positions shall be saved as csv and serve as input to train your model. A *project-material* repository will be provided, containing example functions that can extract body part positions from both existing videos and live camera feeds. To label these movements, we recommend using ELAN. The classifier shall be integrated in the Reveal.js slideshow framework. An example for the slideshow is also provided in the *project-material* repository.

The final application must be fully functional, allowing users to control a slideshow through gestures and be deployable on a workstation in the HCI lab. Further details and explanations will be covered in the corresponding exercise.

The **group size for the project shall be three people** to ensure tasks can be distributed effectively. Master's and bachelor's students may not be in the same group.
If you're unable to find enough team members to reach three or if someone drops out, you need to inform the supervisors. They will work on finding a solution so that you can still form a team of three.

Handover of the project

All results of the practical work and all data related to activities conducted for the project must be stored within the team's assigned GitLab group.

- You have to have one repository named **orga** in your GitLab group. The Readme.md of this project has to specify where resources that shall be considered for grading can be found. It should also be noted which requierements were made by which persons.
- There must be a Readme.md for each repository in your GitLab group that (at a very least) explains what the repository is for.
- All code as well as the *teaser video* must be checked into a separate repository inside your team's GitLab group; the Readme.md has to explain how the code can be set up and how the two evaluation modes (M3,

Processing math: 100%

- Use the git tag *final* for the commits in your repositories you want to submit for the presentation
- **For the WS2025 the deadline for submission via Gitlab and the Git tag final is March 13, 2026, at 11:59 p.m.**

Schedule of the Presentation

The result of the project must be presented and demonstrated. Details about the session schedule will be announced via *WueCampus2*. **The presentations in WS2025 will take place between March 16 and 27, 2026.** One session consists of:

- 5 min organisation
- 5 min score acquisition
- 10 min presentation
- 15 min demonstration and group questioning
- 45 min individual questioning (~15 min per person)
- 15 min debriefing

We expect you to be ready to start at the beginning of your exam. This means that you should be in WhiteLab 1 (Building 50, 00.008) or WhiteLab 2 (Building 50, 00.007) about **15 minutes before the start and set up your workstation** so that your application and presentation are running. **If the exam can not be started or the applications are not working 10 minutes after the start time, the exam must be terminated.**

Grading

The final grade consists of **three components**, which are explained in detail below:

- **Mandatory Requirements:** They have to be fulfilled in order to pass. All mandatory requirements combined are rated with **up to** 60 points.
- **Optional Requirements:** These may be completed to gain additional points. The maximum number of optional requirement points that count toward your grade depends on your team's maximum possible points. For example**,** a team of bachelor students can gain up to 40 points from optional requirements. If this team worked on 10 optional requirements worth 5 points each, the team must choose eight of them that will count toward the grading. The team must **declare** these eight **selected optional requirements in their orga repository in advance**.
- **Individual factor:** Each team member will be questioned individually for 15 minutes about the project. This assesses your comprehension of the implemented code and the underlying methods and showcases contribution to the project. Each person receives an individual factor between 0.4 and 1.1.

Processing math: 100%

Maximal Total Project Points

Depending on the degree program and the group size, the total number of points required to achieve the maximum score varies. This required score is always normalized using a scaling factor so that the final score is capped after the scaling at 100 points. For example, a team of three master's students earned 86 points. Since the required maximum is 110 points, the scaling factor is 100/110. The *scalled total project points* is therefore 78 points: 86 × (100/110) = 78. **Important: If your group has fewer than three members, you must inform your supervisor at least two weeks in advance in order to receive the corresponding bonus.**
Regardless of how many total project points are needed for a grade of 1.0, all mandatory requirements must always be fulfilled. Any remaining points must be obtained through optional requirements. If you complete more optional requirements than necessary, you must document in your Orga repository which optional tasks should be graded.

Master's Program

- Team of three: 110 total points; max. 50 optional points; scaling factor: 100/110
- Team of two: 100 total points; max. 40 optional points; scaling factor: 100/100
- Individual (team of one): 85 total points; max. 25 optional points; scaling factor: 100/85

Bachelor's Program

- Team of three: 100 total points; max. 40 optional points; scaling factor: 100/100
- Team of three: 90 total points; max. 30 optional points; scaling factor: 100/90
- Team of three: 75 total points; max. 15 optional points; scaling factor: 100/75

Final Project Grade Calculation

The project score is the sum of all points from the mandatory and selected optional requirements, this total project points will be multiplied by the scaling factor and after that multiplied by the individual factor.

total project points = mandatory points + optional points
*scalled total project points = total project points * scaling factor*
individual points = individual_factor ∗ scaled total project points
grade <= Marking Scale(individual_points)

Example: If the bachelor team achieves 49 mandatory points and 33 optional points, the total project score is 82 points and the scalled projext points is 82 (= 82*100/100). Each team member then receives an individual grade, calculated by multiplying the scalled total project points with their personal individual factor. This results in their individual points, which are then used to determine the final grade.

team members	mandatory points	optional points	total project points
Rick	49	33	82
Morty	49	33	82
Be	49	33	82

Processing math: 100%

Example: If the master team achieves 55 mandatory points and 42 optional points, the total project score is 97 points and the scalled projext points is 88 (= 88*100/110). Each team member then receives an individual grade, calculated by multiplying the scalled total project points with their personal individual factor. This results in their individual points, which are then used to determine the final grade.

team members	mandatory points	optional points	total project points
Rick	55	42	97
Morty	55	42	97
Beth	55	42	97

Marking Scale

Points	Grade
≥ 95	1.0
≥ 90	1.3
≥ 85	1.7
≥ 80	2.0
≥ 75	2.3
≥ 70	2.7
≥ 65	3.0
≥ 60	3.3
≥ 55	3.7
≥ 50	4.0
< 50	5.0

Individual Score

Each team member receives an individual assessment of their performance during the project presentation. The purpose of the individual questioning is to assess your comprehension of the implemented code and the underlying methods and to showcase your contribution to the project. The questioning focuses on the core aspects of the implementation—feature extraction, the neural network, and the training, validation, testing, and diagnostic activities.

The individual examination is divided into three thematic blocks, each covering different areas of the project. Ea five minutes and will not be extended. During these blocks, students must be able to

Processing math: 100%

identify and show relevant source code, explain specific code passages, and discuss the theoretical foundations on which their implementation is based.

The performance of each team member is evaluated using an individual factor ranging from **0.4 to 1.1**. This evaluation considers several aspects:

- the correctness of the answers
- the time a person needs to respond
- how often questions need to be repeated or reformulated

In addition to the mandatory requirements—which are the most important and must be comprehended by every team member— there must be at least one team member per optional requirement who can explain it. The "optional requirement comprehension" must be distributed evenly across all members to ensure that the workload and responsibility are shared fairly.

Mandatory Requirements [up to 60 points]

Mandatory M1 - M5 have to be fulfilled in order to pass and result in a total of 30 points. M6 and M7 must also be completed, but are scored according to the team's performance.
All Mandatory Requirements have to be worked on by all team members and every team member has to be able to explain their theory and implementation.

M1: Implementation of a neural network with Python and Numpy suitable for gesture classification.

M2: Detection of the mandatory gestures *swipe right*, *swipe left,* and *rotation* using one neural network (see *M1*). The provided validation dataset in wuecampus contains samples for theses mandatory gestures. The mandatory gestures are defined as follows. It is very important to perform these gestures exactly as shown (see also the example videos in wuecampus):

- rotation gestures R_t (right arm, clock wise)
- swipe left gestures SL_t (right arm, from right to left)
- swipe right gestures SR_t (left arm, from left to right)

Note: *The subjects in the test videos may perform other gestures than rotate or swipe - your model should ignore these during the evaluation phase.*

M3: There must be a prediction mode that allows users to control a *reveal.js* webpage via detected gestures. All gesture from M2 as well as all optional gestures have to be detected and used.

M4: Your project has to be deployable on a workstation using git and pip or pipenv. There has to be a pipenv or requirements.txt in the repos listing all libraries used in the project.

Processing math: 100%

M5: Creation of a *teaser video* (1-2 min) showcasing the prototype.

- Make an interesting teaser about your project! If you give your consent, we will publish the video on the chair's YouTube channel. You may also deny that without any consequence, of course.
- The HCI chair's video template has to be used for the intro and outro.
- The video has to be encoded using the *H264* or any other *MPEG-4* codec and has to be stored using the *mp4* container format.

M6: Performance Score [up to 20 points]

Your application must provide a test mode to evaluate the performance requirements. This test mode must be fully executeable regardless of the level of performance achieved. If the test mode is not fully executed, the mandatory requirement will not be met and the examination will be considered unsuccessful.

- Provide a script that reads a given video-transcript (CSV format).
- The provided video transcript was generated from a video that you will not receive and is only used for the exam. The video transcript has the same format as the file *demo_data/demo_video_rotate_frames.csv* in the *project-materials*.
- The script registers the events that would normally be sent to the slideshow (M3): For each frame your application has to add an entry to a new column "events" with either the detected mandatory gesture's name (see M2) or the string "idle".
- While your application iterates over each frame and produces the events only the data of the current and past frames must be considered, never future frames.
- For each gesture in the video-transcript only one event must be registered.
- Write the "events" column into a new file named performance_results.csv (you will be provided with an example file and script for reference).

The score as described in the following will be calculated for this video:

1. When your code correctly detects a gesture (True Positive) and registers an event, you get one mark.
2. For each wrong (False Positive), undetected (False Negative), or redundant event, you get a malus.

The total score is computed like this: $\text{Total Score} = \frac{\text{collected marks} - \text{collected mali} * 0.2}{\text{achievable marks}}$

The total score will have a bonus of 0.1 added to it and then be multiplied by 20 points. The result will be rounded up to determine the final points, with a minimum of 0 points and a maximum of 20 points.

Example:

Let's say in the test video there are a total of 15 gestures, so you can achieve a total of 15 marks if your code registers each of the 15 events correctly. Now, your algorithm misses the first gesture and confuses the second gesture with another gesture but nails the remaining 13 gestures. This means your algorithm achieved 13 marks, and 3 mali (2 for missing the first two gestures and 1 for registering a wrong gesture). The score now amounts to $\frac{13 - 3 * 0.2}{15} = 0.83$. A bonus of 0.1 is added to your score of 0.83. This means, you get $(0.83 + 0.1) * 20 = 18.6 \rightarrow 19$ points (decimals will be rounded up).

You'll be provided with the algorithm for the determination of the evaluation score so you can test it on the validation videos and verify how well it works (see the *project-material* repository).

Processing math: 100%

M6.1 Bonus [up to +5 points]

There is also a bonus available, which can add up to an additional 5 points. Besides the test video we use to determine your performance score, there is also a bonus video. This video contains more difficult gestures and really tests the precision of your application. The bonus performance score will be calculated the same as described in M6 (without any bonus percentage points).

M7: Presentation of your Approaches towards Data Preparation, Hyperparameter Choices, and Result Evaluations [up to 10 points]

The objective for you here is to demonstrate within a 10 min presentation that you have taken a methodical approach making the various decisions that have led to your final machine learning model. In addition to the presented methodology for data preparation, hyperparameter search and result evaluations, the quality of the presentation is also considered in the grading.

- Show how you have analysed your data and found/built appropriate features – justify your decisions!
- Show and visualize the training metrics, i.e., learning curve and F1 score history.
- Describe your hyperparameter optimization efforts and show as well as visualize the results of your analyses that drove your decisions towards a better model.
- Remember that we can only consider anything that is versioned inside your GitLab group for grading; so make sure that you push any code you use to create visualizations and other analyses for your presentation, because we may ask you about that code!

Optional Requirements

Note that you can train dedicated models for these Optionals, so you don't have to reuse the same model you are using for the Mandatories.
O3 - O8 require the detection of *additional* gestures and according mapping to actions on the reveal.js slideshow. Your application needs to support all gestures you opt for plus M2 *in the same run*.
O1.1 are extensions of O1 and can only be fulfilled if O1 is done.
Each option selected must be fully explained by at least one person. All options selected must be divided equally among the team.

O1: Machine Learning Framework Package [up to 15 points]

The idea of this requirement is for you to demonstrate that you are able to abstract away code that is specific to machine learning (e.g., forward/backpropagation, gradient descent, etc.) from code that is specific to your project (MNIST, gesture detection, etc.).

- Organize your code dedicated to machine learning in general as a dedicated Python package.
- The framework must support the creation and training of neural networks for general classification tasks.
- API – you may recreate the syntax of popular Python frameworks like PyTorch or Keras.

Processing math: 100%

- Maintain the package in a dedicated GitLab repository inside your group.
- The repository **must have a Readme.md that documents the usage of your framework**; in the exam you will be asked to show your framework using the Readme.md, so make sure it's well structured, concise, and that it documents all functionality so you can be sure everything gets considered for grading.
- The repository must have an **executable example python script or notebook** that demonstrates the usage of your framework with a simple example (i.e., dataset from the midterm submission like MNIST). It has to offer the possibility to customize the number of layers, hidden size per layer, cost function, and all further hyperparameters. It also has to offer a prediction mode that allows to do a prediction based on the previously trained model with some unseen data.
- Use this framework for your main project (training *and* prediction). The framework has to be integrated to the main project using pip. There should not be a manual copy of the framework code to the main project.
- The framework has to cover at least the following modules:
  - Neural Network Layer
  - Cost Functions
    - Mean Squared Error
    - Cross Entropy (with Softmax)
  - Optimizers
    - Gradient Descent

O1.1 Visualization [up to 5 points]

- Implement various functions within the framework to visually compare the results of multiple trained models.
- All functionality should be included in the example notebook from the framework and be **documented in the accompanying Readme.md.**
- After training, visualize at least the following metrics: Loss, accuracy, and F1 score across training times, and generate confusion matrices. Ensure all visualizations are saved.
- Additionally, store the raw data used to create these graphs.
- Provide functionality to load the saved data from multiple models to generate combined visualizations. These should display loss, accuracy, **or** F1 score across training times for each model in a distinguishable manner.

O2: Implementation and Application of PCA [up to 5 points]

Employ the principal component analysis:

- Implement the PCA and use it appropriately.
- Document and visualize your efforts in the presentation (M7).

O12: Implementation and Application of Gradient Descent Variations [up to 5 points]

- Implement two different gradient descent variations (e.g., momentum-based GD, NAG) in addition to the vanilla GD and use them appropriately.
- Document and visualize your efforts and compare the GD variations to the vanilla GD in the presentation (M7).

Processing math: 100%

O3: Additional gestures: *Rotate* right with right hand counter clock wise [up to 5 points]

O4: Additional gestures: P*inch/spread* [up to 5 points]

O5: Additional gestures: S*wipe up/down* both with same hand [up to 5 points]

O6: Additional gestures: F*lip table* both hands up [up to 5 points]

O7: Additional gestures: *Point* [up to 5 points]

O8: Additional gestures: S*pin* both hands in front of chest, rotating around each other [up to 5 points]

O9: Detection of whether the user is looking at the presentation or not and appropriate blocking of unintended gesture commands. **[up to 5 points]**

O10: Control a game (e.g., Tetris) with gestures. [up to 5 points]
The game must be well playable with at least three gestures, you may use a dedicated neural network for this. For grading the ratio between the number of gestures and how well the game can be played will be considered. For example, Tetris with three gestures that can be played well gets the same grading as another game that makes use of five gestures that might not work as well individually.

O11: Identify humans instead of gestures [up to 5 points]
Train your network to classify members of your team in prediction mode. This means that it still receives the same inputs as before (motion data from mediapipe), but the classes are different (e.g., "Anna", "Otto", and "Miriam" instead of "swipe left", "rotate", etc.). This way, your model should be able to identify the user based on their motions, an actively discussed topic in research! Think about a good way to display who the model thinks it is currently seeing in prediction mode. This Optional only makes sense, of course, if your team consists of at least two members.

Note: *You can propose ideas for additional optional requirements. To do so, present the idea to Lukas Schach via e-mail at least two weeks before the final presentation week.*

Software Specifics

If you would like to use software or packages that are not listed under this section, ask the lecturers.
Allowed Software:

- Python >=3.6
- HTML, CSS, JavaScript
- Mediapipe

Processing math: 100%

Python Libraries

Whitelist: The following libraries are allowed to be used by submitted code:

- Python standard libraries
- Numpy
- Pandas
- matplotlib
- tqdm
- jupyter
- pyyaml
- sanic
- socketio

Blacklist: In general, no library is allowed that provides machine learning functionalities. The following list states libraries that are forbidden in any case. Importing or using any of the following libraries or other code that provides funcationality for any of the requirements listed above leads to an overall rating of 5.0 (failed).

- Tensorflow
- PyTorch
- Keras
- scikit-learn
- Tensorboard
- scipy
- scikit-optimize
- AutoML

Last modified: Wednesday, 3 December 2025, 3:44 PM

◄ Validation Videos

Jump to...

Impressum | Kontakt | Datenschutzerklärung - WueCampus | Erklärung zur Barrierefreiheit | Bildnachweise

Processing math: 100%